

Distributed Systems

DAT520 - Spring 2026

Chapter 6 - Consensus Variants

Prof. Hein Meling

Consensus

Classic consensus:

Processes propose values

→ One value is decided

Module 5.1: Interface and properties of (regular) consensus

Module:

Name: Consensus, **instance** c .

Events:

Request: $\langle c, \text{Propose} \mid v \rangle$: Proposes value v for consensus.

Indication: $\langle c, \text{Decide} \mid v \rangle$: Outputs a decided value v of consensus.

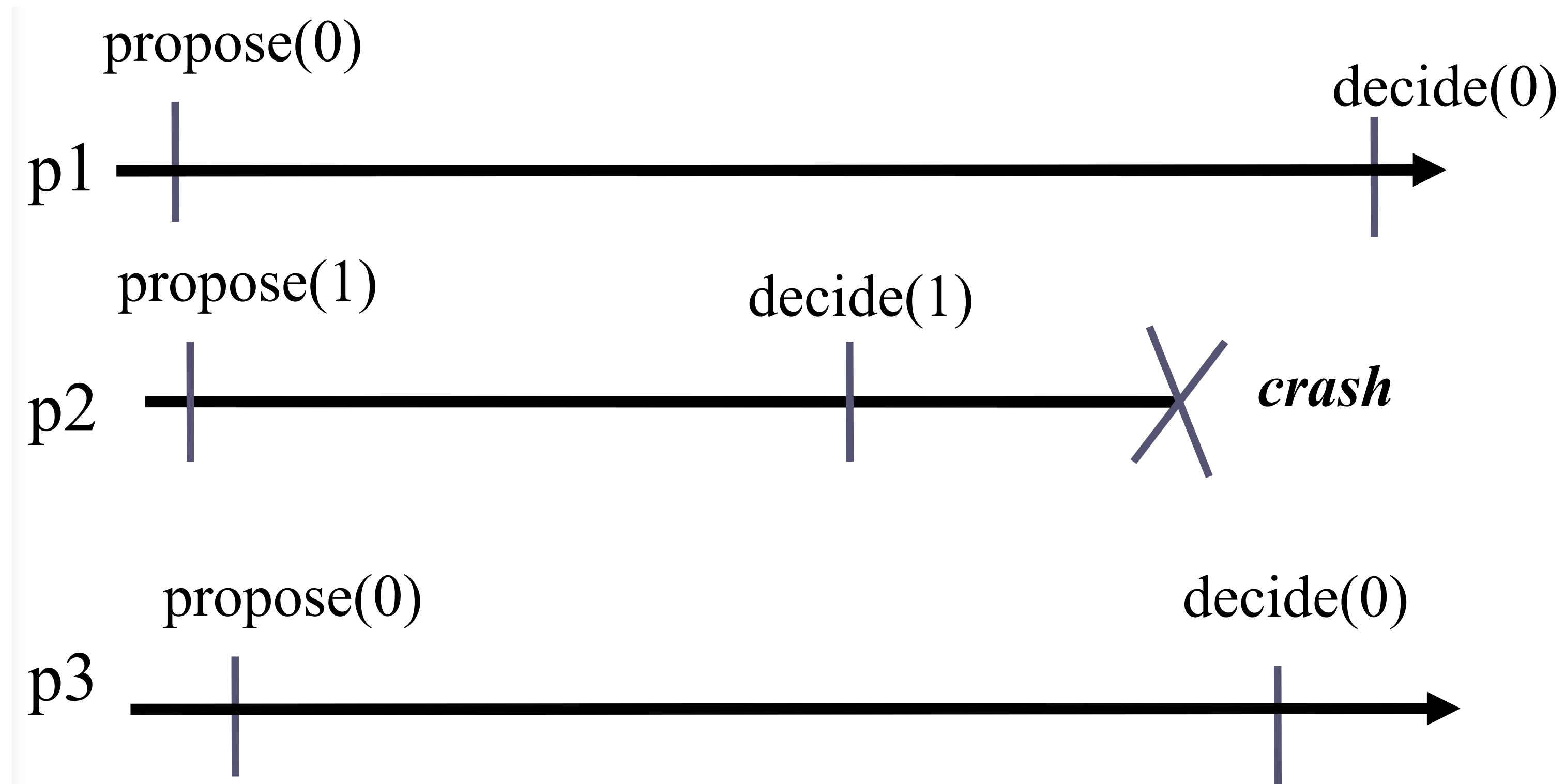
Properties:

C1: Termination: Every correct process eventually decides some value.

C2: Validity: If a process decides v , then v was proposed by some process.

C3: Integrity: No process decides twice.

C4: Agreement: No two correct processes decide differently.



Module 5.2: Interface and properties of uniform consensus

Module:

Name: UniformConsensus, **instance** *uc*.

Events:

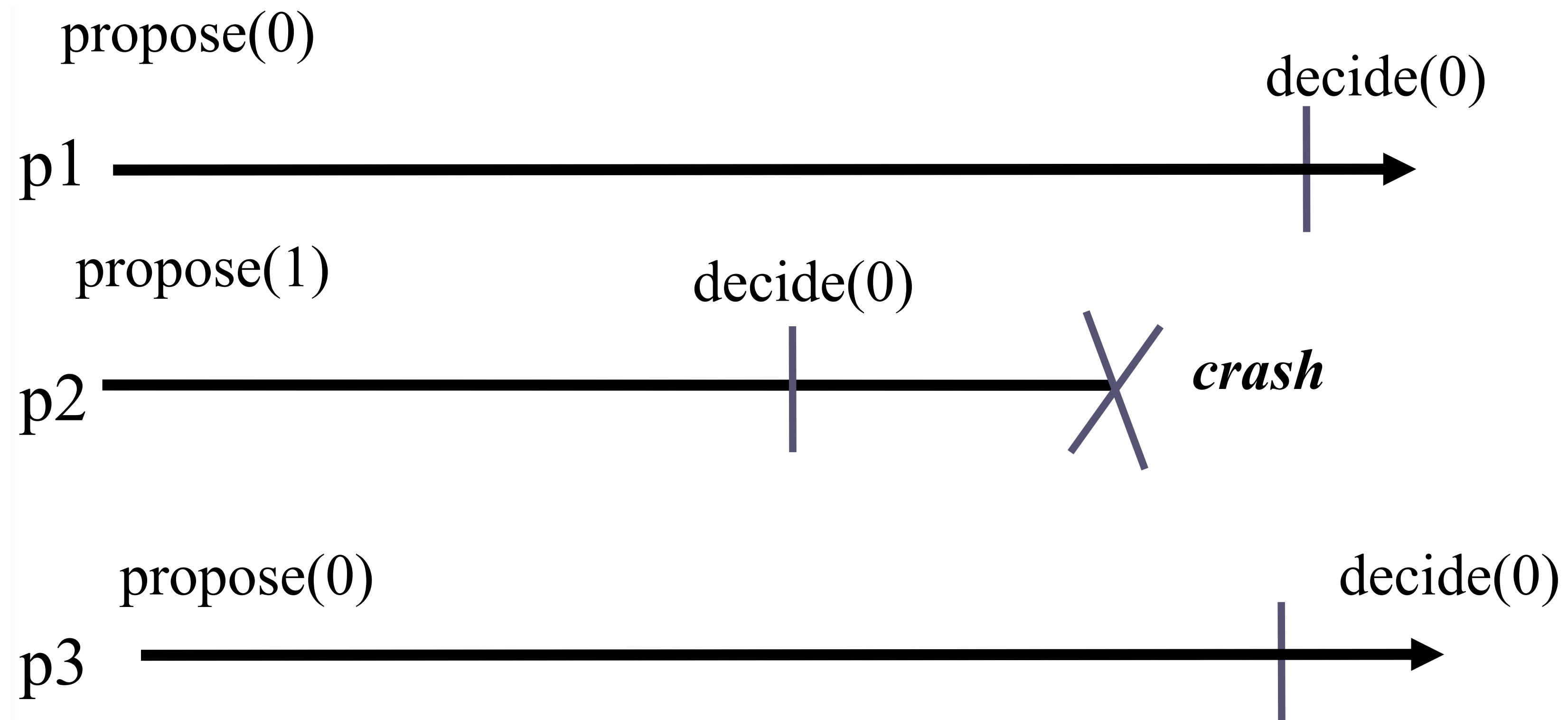
Request: $\langle uc, Propose \mid v \rangle$: Proposes value v for consensus.

Indication: $\langle uc, Decide \mid v \rangle$: Outputs a decided value v of consensus.

Properties:

UC1–UC3: Same as properties C1–C3 in (regular) consensus (Module 5.1).

UC4: *Uniform agreement*: No two processes decide differently.



Motivation for Consensus Variants

Many applications need stronger coordination guarantees

- Replication → Total Order Broadcast
- Distributed Databases → (Non-blocking) Atomic Commit
- Group Membership
- View-Synchronous Broadcast

Key Observations

- In several variants:
 - Decisions cannot be arbitrary proposed values
- Instead, they must:
 - Obey coordination constraints
 - Satisfy upper-layer application semantics
- Consensus is a **building block**, not the final abstraction

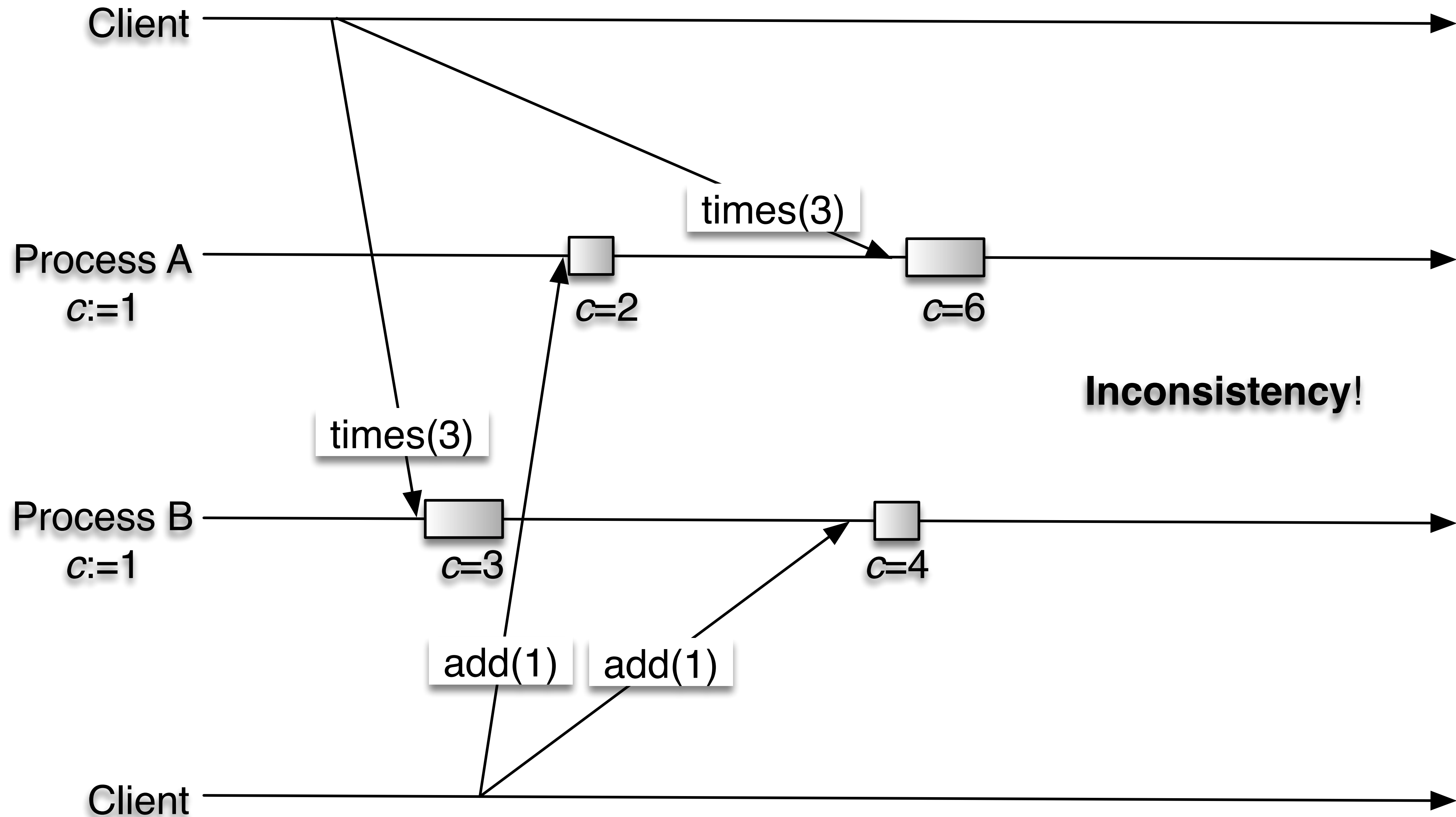
Total Order Broadcast

Ordering Broadcast Messages

- We have seen:
 - **FIFO Broadcast** → preserves sender order
 - **Causal Order Broadcast** → preserves causality
- Causal relation
 - If message m_2 depends on m_1 → all processes must deliver m_1 before m_2
- Messages that are not causally related are **concurrent**

Limitation of Causal Broadcast

- CO-Broadcast guarantees:
 - Order for causally related messages
- But:
 - **No ordering** among **concurrent** messages.



Total Order Broadcast

- **All processes deliver messages in the same order**
- Stronger than:
 - Reliable Broadcast (same set)
 - Causal Broadcast (partial order)
- Now we require:
 - Agreement on the **same sequence**

Module 6.1: Interface and properties of regular total-order broadcast

Module:

Name: TotalOrderBroadcast, **instance** *tob*.

Events:

Request: $\langle tob, Broadcast \mid m \rangle$: Broadcasts a message m to all processes.

Indication: $\langle tob, Deliver \mid p, m \rangle$: Delivers a message m broadcast by process p .

Properties:

TOB1: Validity: If a correct process p broadcasts a message m , then p eventually delivers m .

TOB2: No duplication: No message is delivered more than once.

TOB3: No creation: If a process delivers a message m with sender s , then m was previously broadcast by process s .

TOB4: Agreement: If a message m is delivered by some correct process, then m is eventually delivered by every correct process.

TOB5: Total order: Let m_1 and m_2 be any two messages and suppose p and q are any two correct processes that deliver m_1 and m_2 . If p delivers m_1 before m_2 , then q delivers m_1 before m_2 .

Total Order Broadcast

- TOB5:
 - $p.\text{deliver}(m1) \rightarrow p.\text{deliver}(m2) \Rightarrow q.\text{deliver}(m1) \rightarrow q.\text{deliver}(m2)$

Total Order Broadcast

- Active Replication = State Machine Replication = Replicated State Machine
- All replicas:
 - Deliver the same commands
 - In the same order
 - Execute deterministically
- $\Rightarrow$ Same state everywhere

Total Order and Causal Order

- Important distinctions:
 - TO does **not** respect causal order
 - CO does **not** impose total order
- Why?
 - TO doesn't care about causal dependencies
 - CO cannot order concurrent messages
- They solve different problems

Total Order and Causal Order: Can They be Combined?

- Yes, but for State Machine Replication:
 - We only need a **total order**
- Causality preservation is unnecessary when:
 - Clients issue concurrent requests
 - We just need a **single global order**

Build Total Order Broadcast Using

- Reliable Broadcast
- Repeated Consensus

Build Total Order Broadcast Using

- Process
 - RB-delivered messages are stored in a set of **unordered messages**
 - Periodically run consensus
 - Consensus decides a set to deliver
 - Sort deterministically
 - Deliver in that order

Algorithm 6.1: Consensus-Based Total-Order Broadcast

Implements:

TotalOrderBroadcast, **instance** *tob*.

Uses:

ReliableBroadcast, **instance** *rb*;
Consensus (multiple instances).

upon event $\langle tob, Init \rangle$ **do**

unordered := $\emptyset$;

delivered := $\emptyset$;

round := 1;

wait := FALSE;

← Round = Paxos Slot

upon event $\langle tob, Broadcast \mid m \rangle$ **do**

trigger $\langle rb, Broadcast \mid m \rangle$;

upon event $\langle rb, Deliver \mid p, m \rangle$ **do**

if $m \notin delivered$ **then**

unordered := *unordered* $\cup \{(p, m)\}$;

upon *unordered* $\neq \emptyset \wedge wait = FALSE$ **do**

wait := TRUE;

Initialize a new instance *c.round* of consensus;

trigger $\langle c.round, Propose \mid unordered \rangle$;

upon event $\langle c.r, Decide \mid decided \rangle$ **such that** $r = round$ **do**

forall $(s, m) \in sort(decided)$ **do**

trigger $\langle tob, Deliver \mid s, m \rangle$;

delivered := *delivered* $\cup \{m\}$;

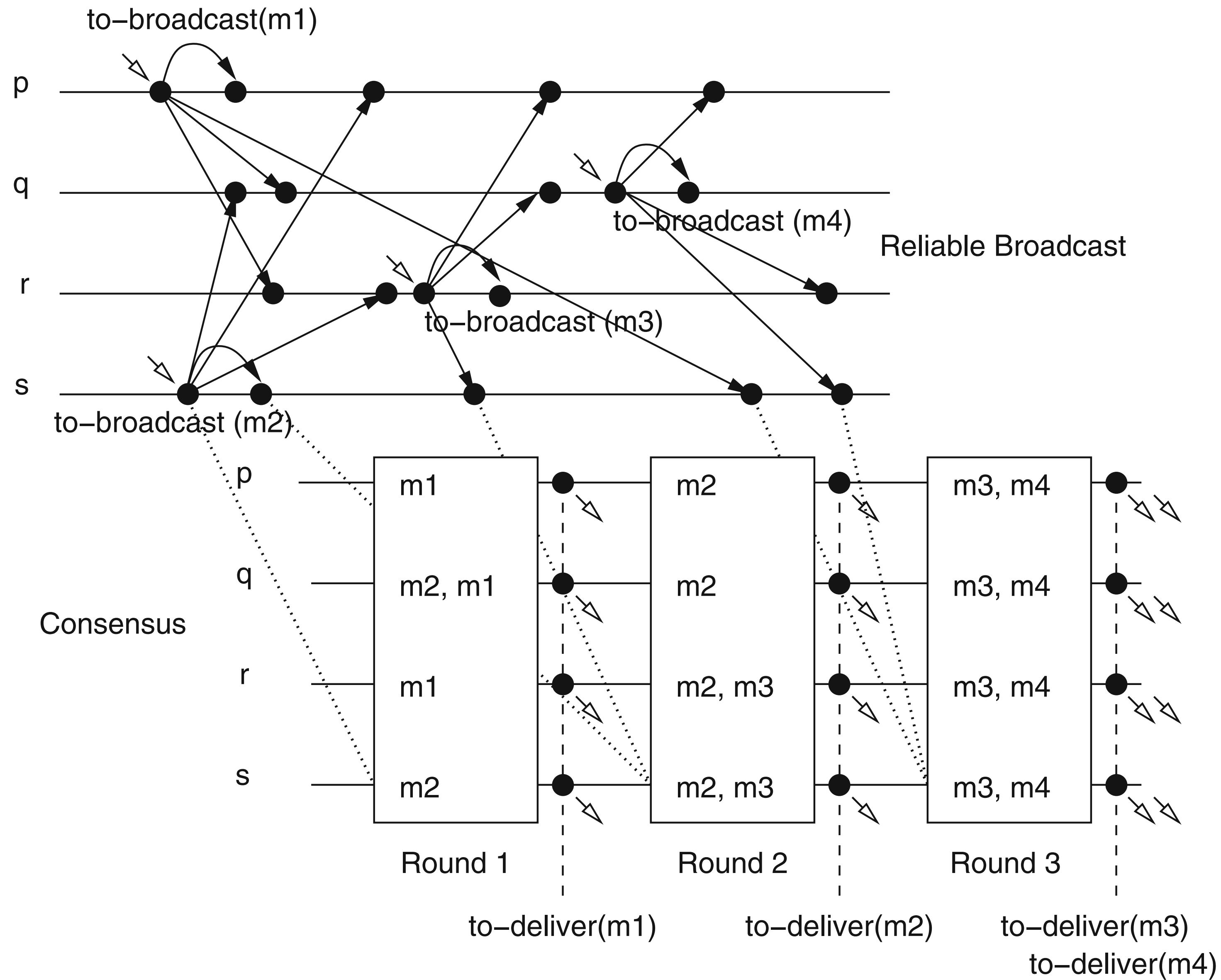
unordered := *unordered* $\setminus \{(s, m)\}$;

round := *round* + 1;

wait := FALSE;

// by the order in the resulting sorted list

← Errata



Terminating Reliable Broadcast

Recall

- Reliable Broadcast guarantees:
 - Validity
 - Agreement
 - No duplication
 - No creation

- But:
 - Reliable Broadcast does not guarantee **termination**
 - A process may **wait forever**

Why do we need Termination?

- *Some applications require a **decision**, not indefinite waiting*
- Example:
 - A designated process **must** broadcast a value
 - Others must **eventually** decide:
 - Deliver the value
 - Or conclude it will never arrive
- We need **closure**

TRB strengthens RB with

- A specific termination property
- Defined for:
 - A **known** broadcaster process s
 - A message m
 - A special **failure value** Δ

TRB Semantics

- Process s is supposed to broadcast message m
 - (where $m \neq \Delta$)
- Other processes must eventually deliver:
 - m : if s is correct
 - Δ : if s crashes
- They must not wait forever

Why RB is not enough

- Consider:
 - Source process s is expected to broadcast m
 - s crashes during the broadcast
- Some processes receive m
- Some do not
- Now what?

Why RB is not enough

- Assume process p :
 - Has not received m
 - Detects that s has crashed
- p faces uncertainty:
 - Should it wait?
 - Maybe another process q will forward m ?
- RB gives no rule to resolve this

Core Issue

- Without termination:
 - p may wait forever
- TRB forces a decision:
 - Either **everyone** delivers m
 - Or **everyone** delivers Δ

Ch 6.3 Terminating Reliable Broadcast

Module 6.4: Interface and properties of uniform terminating reliable broadcast

Module:

Name: UniformTerminatingReliableBroadcast, **instance** $utrb$, with sender s .

Events:

Request: $\langle utrb, Broadcast \mid m \rangle$: Broadcasts a message m to all processes.
Executed only by process s .

Indication: $\langle utrb, Deliver \mid p, m \rangle$: Delivers a message m broadcast by process p or the symbol $\triangle$.

Properties:

UTRB1: *Validity*: If a correct process p broadcasts a message m , then p eventually delivers m .

UTRB2: *Termination*: No process delivers more than one message.

UTRB3: *Integrity*: If a correct process delivers some message m , then m was either previously broadcast by process s or it holds $m = \triangle$.

UTRB4: *Uniform Agreement*: If any process delivers a message m , then every correct process eventually delivers m .

Algorithm 6.3: Consensus-Based Uniform Terminating Reliable Broadcast

Implements:

UniformTerminatingReliableBroadcast, **instance** *utrb*, with sender *s*.

Uses:

BestEffortBroadcast, **instance** *beb*;

UniformConsensus, **instance** *uc*;

PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle \textit{utrb}, \textit{Init} \rangle$ **do**

proposal $:= \perp$;

upon event $\langle \textit{utrb}, \textit{Broadcast} \mid m \rangle$ **do**

trigger $\langle \textit{beb}, \textit{Broadcast} \mid m \rangle$;

upon event $\langle \textit{beb}, \textit{Deliver} \mid s, m \rangle$ **do**

if *proposal* $= \perp$ **then**

proposal $:= m$;

trigger $\langle \textit{uc}, \textit{Propose} \mid \textit{proposal} \rangle$;

upon event $\langle \mathcal{P}, \textit{Crash} \mid p \rangle$ **do**

if $p = s \wedge \textit{proposal} = \perp$ **then**

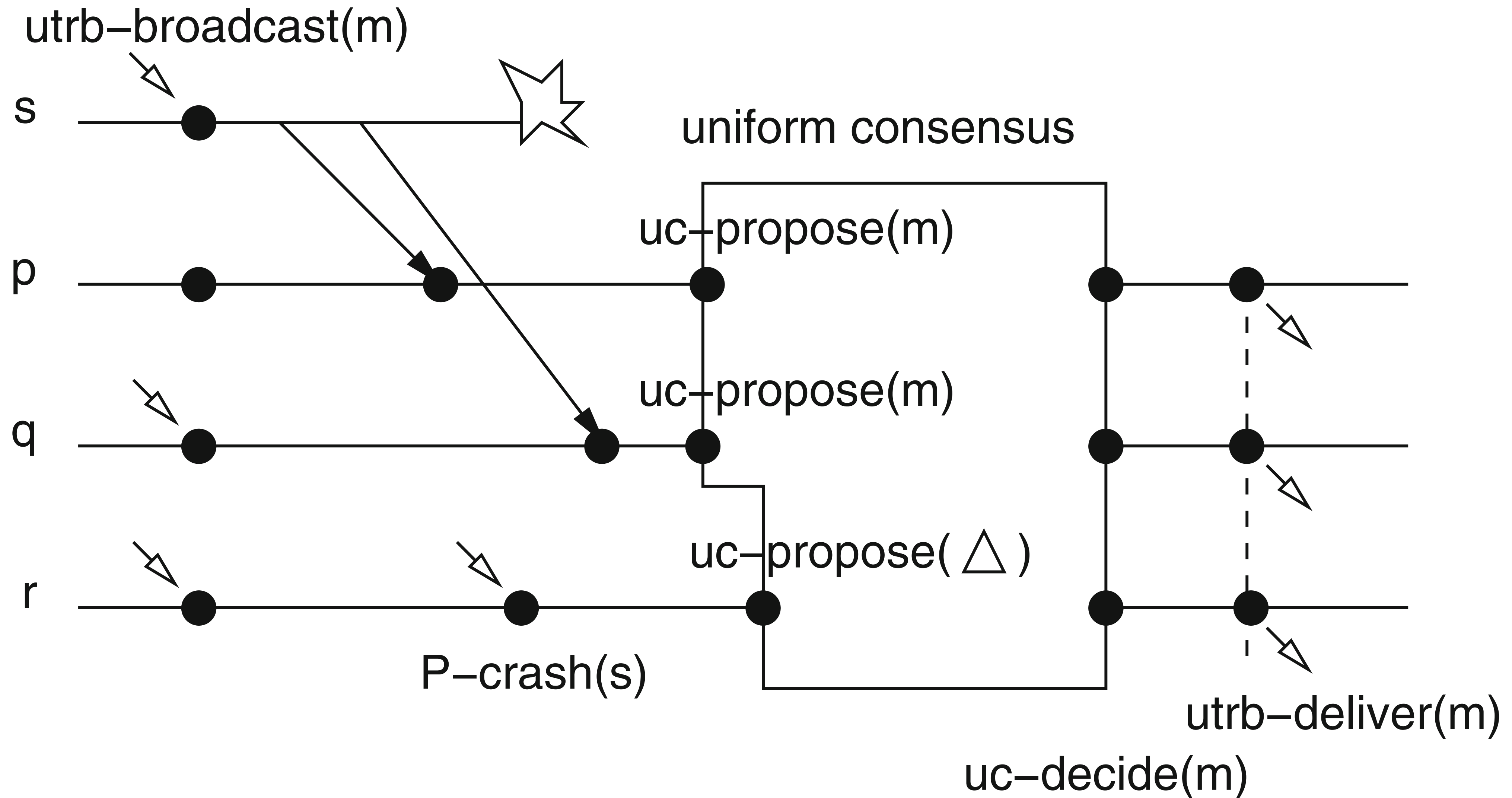
proposal $:= \Delta$;

trigger $\langle \textit{uc}, \textit{Propose} \mid \textit{proposal} \rangle$;

upon event $\langle \textit{uc}, \textit{Decide} \mid \textit{decided} \rangle$ **do**

trigger $\langle \textit{utrb}, \textit{Deliver} \mid s, \textit{decided} \rangle$;

Ch 6.3 Terminating Reliable Broadcast



Core Difference

- TRB requires:
 - A decision, even if the broadcaster crashes
- Unlike RB/URB:
 - Delivery is mandatory
 - No indefinite waiting
- Every correct process eventually delivers **something**

Skipping ahead

- Read on your own
 - 6.2 Byzantine Total-Order Broadcast
 - 6.4 Fast Consensus
 - 6.5 Fast Byzantine Consensus

Non-blocking Atomic Commit

Unit of Work: Transaction

- A **transaction** is:
 - A portion of a program
 - Delimited by begin/end
 - A logical unit of data processing

Goal

- Ensure that a distributed transaction either:
 -  Commits everywhere, or
 -  Aborts everywhere
- No split-brain outcomes

Who Participates in a Distributed Transaction

- Participants and Coordinators (Processes)
- Data Managers = Processes
 - Each data manager controls part of the data
- They must ensure:
 - Either all apply the update
 - Or all discard it
- Agreement is mandatory

Classic Solutions

- Two-Phase Commit (2PC)
- Three-Phase Commit (3PC)
- Both attempt to solve atomic commitment
- But...
 - They may **block** under failures

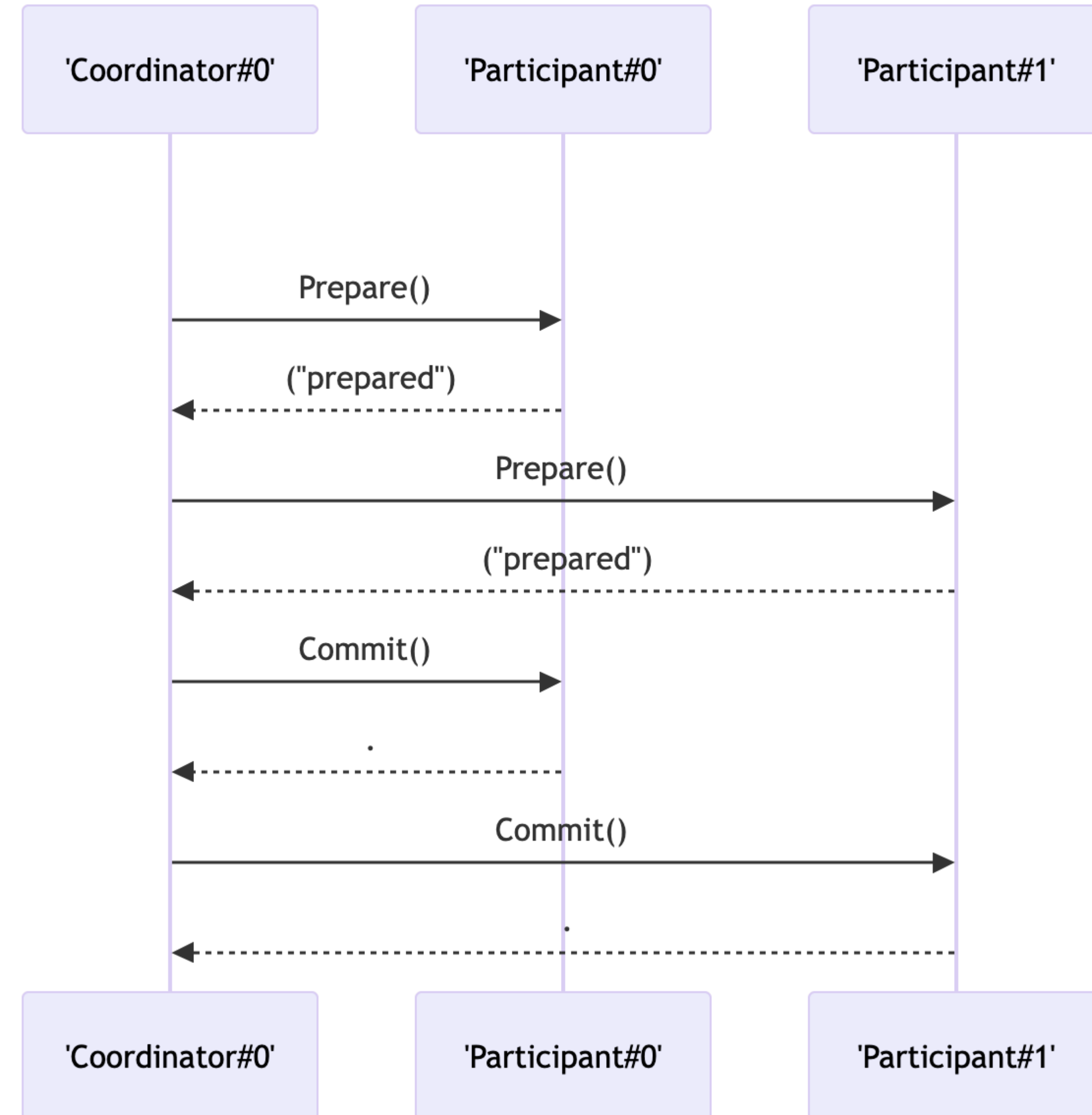
Two-Phase Commit (2PC)

- **Phase 1 — Prepare**

- Coordinator asks: “*Can you commit?*”
- Participants reply:
 - YES (prepared)
 - NO (abort)

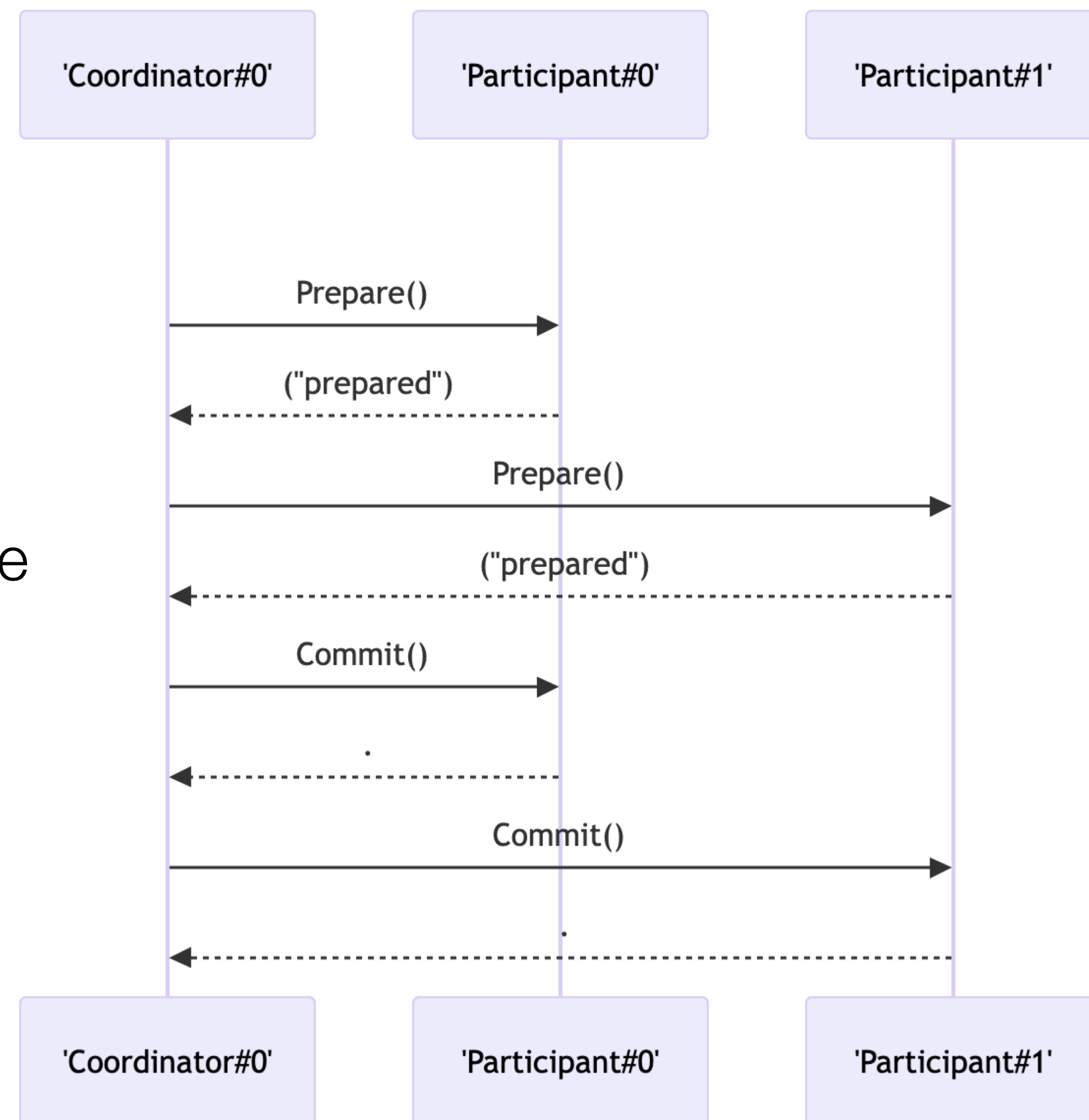
- **Phase 2 — Decision**

- If all YES → COMMIT
- Otherwise → ABORT



Two-Phase Commit (2PC)

- Why 2PC Blocks
- If the coordinator crashes:
 - Participants may be stuck in prepared state
 - They cannot decide safely
 - They must wait
- ⚠ Safety is preserved
- ❌ Liveness is not guaranteed



Three-Phase Commit (3PC)

- Add a phase to avoid uncertainty
- Requires **stronger timing** assumptions

Is NBAC Just Consensus?

- At first glance:
 - Decision $\in \{\text{COMMIT}, \text{ABORT}\}$
 - All must agree to commit
- Looks like binary consensus
- But there is a crucial semantic difference

The Crucial Difference

- Consensus
 - *Can decide either value*
 - As long as one correct process proposed it
- **Atomic Commit**
 - Cannot decide COMMIT
 - if any process proposes ABORT
 - Every participant has **veto power**

Ch 6.6 Non-blocking Atomic Commit

Module 6.7: Interface and properties of nonblocking atomic commit

Module:

Name: NonBlockingAtomicCommit, **instance** *nbac*.

Events:

Request: $\langle nbac, Propose \mid v \rangle$: Proposes value $v = \text{COMMIT}$ or $v = \text{ABORT}$ for the commit.

Indication: $\langle nbac, Decide \mid v \rangle$: Outputs the decided value for the commit.

Properties:

NBAC1: Termination: Every correct process eventually decides some value.

NBAC2: Abort-Validity: A process can only decide ABORT if some process proposes ABORT or a process crashes.

NBAC3: Commit-Validity: A process can only decide COMMIT if no process proposes ABORT.

NBAC4: Integrity: No process decides twice.

NBAC5: Uniform Agreement: No two processes decide differently.

Algorithm 6.6: Consensus-Based Nonblocking Atomic Commit

Implements:

 NonBlockingAtomicCommit, **instance** *nbac*.

Uses:

 BestEffortBroadcast, **instance** *beb*;

 UniformConsensus, **instance** *uc*;

 PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle \textit{nbac}, \textit{Init} \rangle$ **do**

 voted := $\emptyset$;

 proposed := FALSE;

upon event $\langle \mathcal{P}, \textit{Crash} \mid p \rangle$ **do**

 if *proposed* = FALSE **then**

 trigger $\langle \textit{uc}, \textit{Propose} \mid \text{ABORT} \rangle$;

 proposed := TRUE;

upon event $\langle \textit{nbac}, \textit{Propose} \mid v \rangle$ **do**

 trigger $\langle \textit{beb}, \textit{Broadcast} \mid v \rangle$;

upon event $\langle \textit{beb}, \textit{Deliver} \mid p, v \rangle$ **do**

 if $v = \text{ABORT} \wedge \textit{proposed} = \text{FALSE}$ **then**

 trigger $\langle \textit{uc}, \textit{Propose} \mid \text{ABORT} \rangle$;

 proposed := TRUE;

 else

 voted := *voted* $\cup \{p\}$;

 if *voted* = $\Pi \wedge \textit{proposed} = \text{FALSE}$ **do**

 trigger $\langle \textit{uc}, \textit{Propose} \mid \text{COMMIT} \rangle$;

 proposed := TRUE;

upon event $\langle \textit{uc}, \textit{Decide} \mid \textit{decided} \rangle$ **do**

 trigger $\langle \textit{nbac}, \textit{Decide} \mid \textit{decided} \rangle$;

In Summary

- In consensus:
 - Minority values can be ignored
- In NBAC:
 - A single abort veto cannot be ignored
- NBAC =
 - Binary consensus + veto semantics

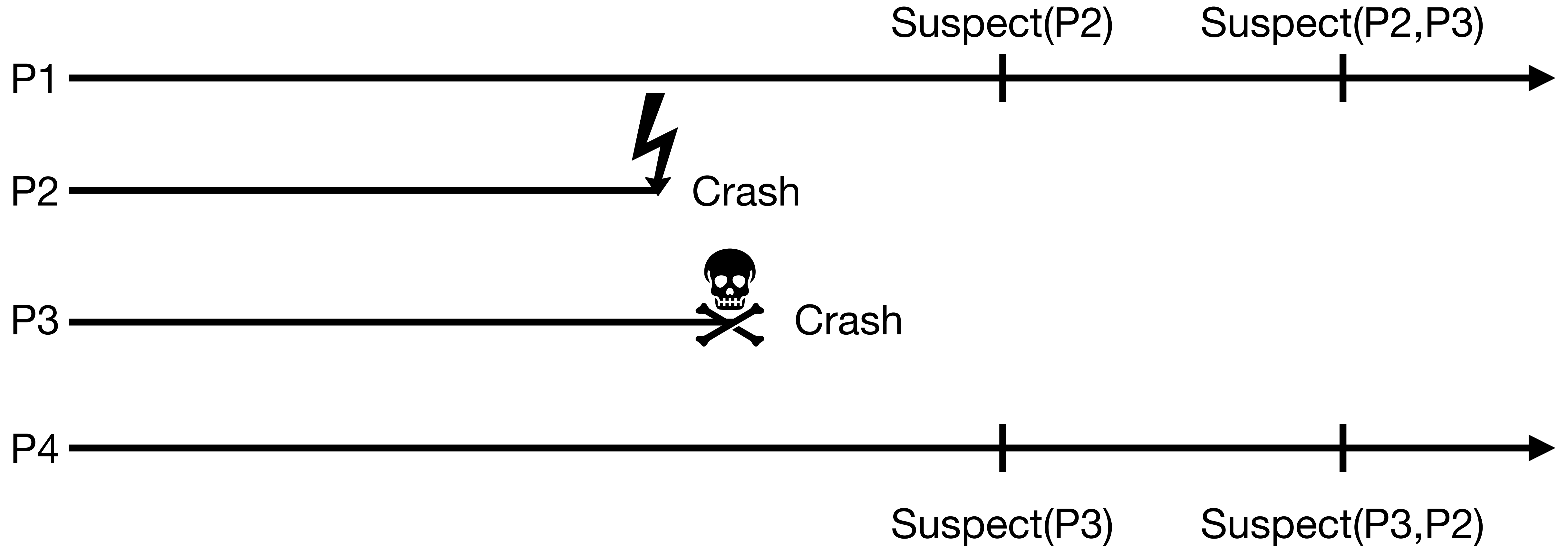
Group Membership

From Failure Detectors to Group Membership

So Far

- Each process uses a Failure Detector (FD)
- Each process maintains its **own local view** of who is correct
- These views:
 - May be **inaccurate**
 - Are **not coordinated**
 - May *observe* **crashes** in **different orders**

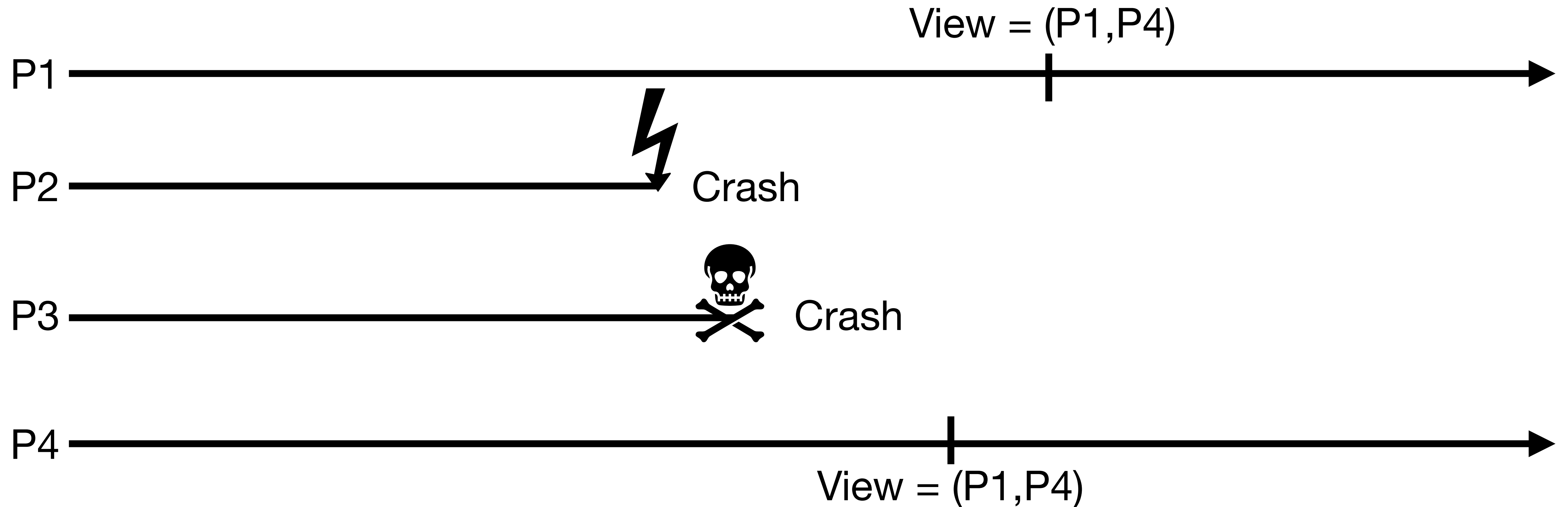
Perfect Failure Detector



Problem

- Processes may observe
 - Different views of the system's evolution
- No guarantee of consistent exclusion of crashed processes
 - ➡ We need a **stronger abstraction**

Perfect Failure Detector



The Group Membership Abstraction: Goal

- Provide consistent information about which processes
 - have **crashed**
 - are **still members** of the group
- Also coordinates:
 - *Joining* and *leaving* (but here: focus on crashes only)

The Group Membership Abstraction: Definitions

- A **group** = set of processes
- The current membership = the **view**

$$V_i = (i, \mathbf{M}_i)$$

- Where:
 - i = unique view identifier
 - $\mathbf{M}_i$ = set of member processes

The Group Membership Abstraction: Views and Installation

- **Initial view:** $V_0 = (0, P_i)$
 - where P_i is the initial set of processes.
- **Installing a view:** A process that delivers a view V_i is said to:
 - Install view V_i

The Group Membership Abstraction: Monotonicity

- All processes install the **same sequence of views**:
 - $V_0 = (0, \mathbf{M}_0)$, $V_1 = (1, \mathbf{M}_1)$, $V_2 = (2, \mathbf{M}_2)$, ...
- With:
 - $\mathbf{M}_{i+1} \subseteq \mathbf{M}_i$
-  Views shrink over time (crash-only model)

Module 6.8: Interface and properties of group membership

Module:

Name: GroupMembership, **instance** gm .

Events:

Indication: $\langle gm, View \mid V \rangle$: Installs a new view $V = (id, M)$ with view identifier id and membership M .

Properties:

GM1: Monotonicity: If a process p installs a view $V = (id, M)$ and subsequently installs a view $V' = (id', M')$, then $id < id'$ and $M \supseteq M'$.

GM2: Uniform Agreement: If some process installs a view $V = (id, M)$ and another process installs some view $V' = (id, M')$, then $M = M'$.

GM3: Completeness: If a process p crashes, then eventually every correct process installs a view (id, M) such that $p \notin M$.

GM4: Accuracy: If some process installs a view (id, M) with $q \notin M$ for some process $q \in \Pi$, then q has crashed.

Consensus-based Group Membership: Code Idea

- When a crash is detected:
 - Trigger a **consensus instance**
 - Agree on which process to exclude
 - **Install new view**

Algorithm 6.7: Consensus-Based Group Membership

Implements:

GroupMembership, **instance** gm .

Uses:

UniformConsensus (multiple instance);
PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle gm, Init \rangle$ do

$(id, M) := (0, \Pi)$;
 $correct := \Pi$;
 $wait := FALSE$;
trigger $\langle gm, View \mid (id, M) \rangle$;

upon event $\langle \mathcal{P}, Crash \mid p \rangle$ do

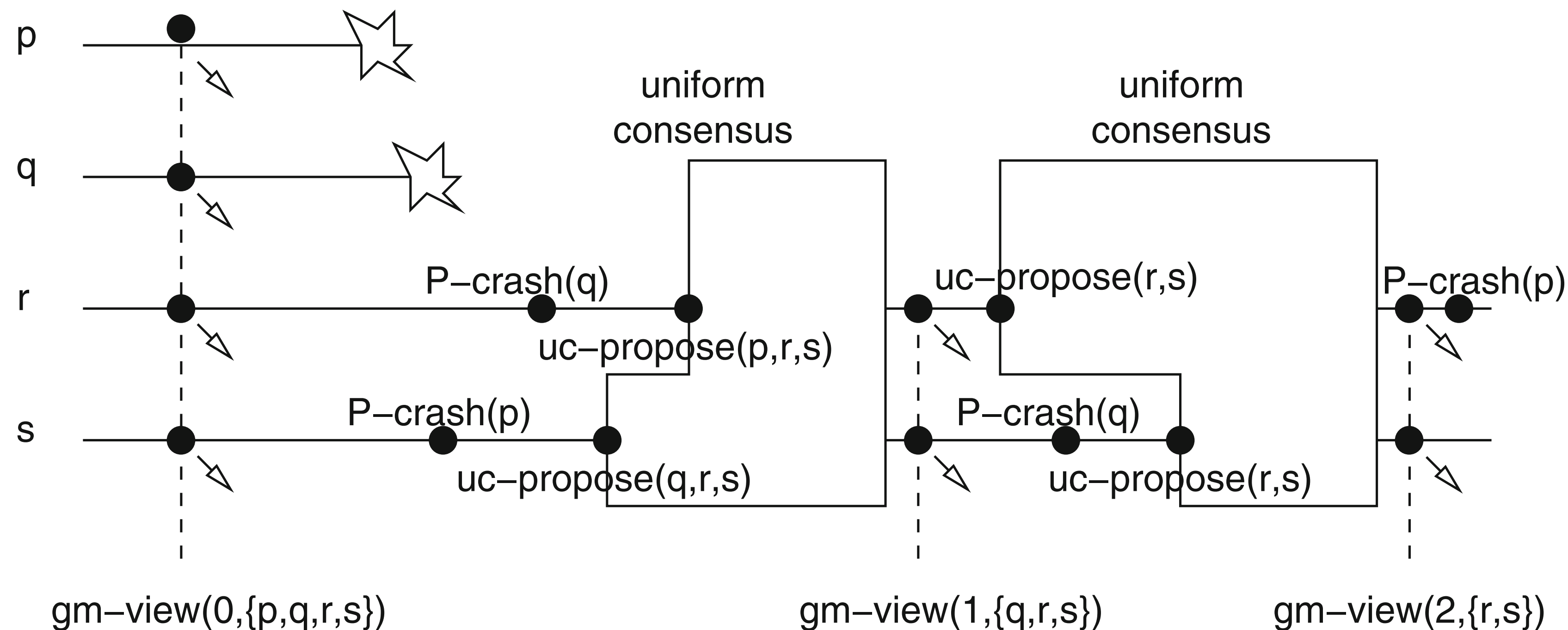
$correct := correct \setminus \{p\}$;

upon $correct \subsetneq M \wedge wait = FALSE$ do

$id := id + 1$;
 $wait := TRUE$;
Initialize a new instance $uc.id$ of uniform consensus;
trigger $\langle uc.id, Propose \mid correct \rangle$;

upon event $\langle uc.i, Decide \mid M' \rangle$ such that $i = id$ do

$M := M'$;
 $wait := FALSE$;
trigger $\langle gm, View \mid (id, M) \rangle$;



Conceptual Takeaway

Failure Detector:

“I suspect someone crashed.”

Group Membership:

“We all agree that this process is no longer part of the system.”

Group Membership

All aboard!

View V1: {p, q, r}

NEW VIEW V2: {q, r}

CRASH!

Help!
Blub...
Blub...

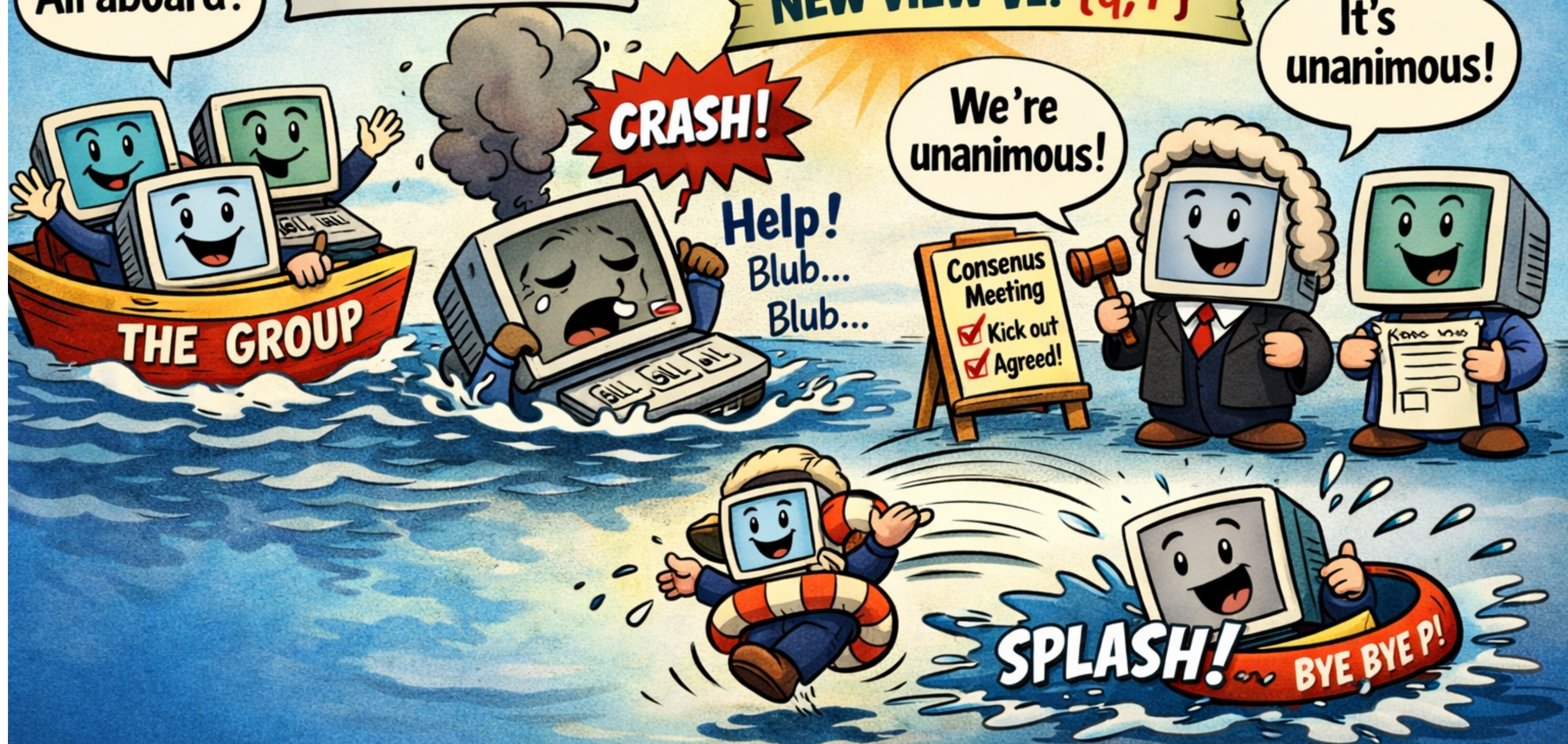
We're
unanimous!

It's
unanimous!

Consensus
Meeting
☒ Kick out
☒ Agreed!

SPLASH!

BYE BYE P!

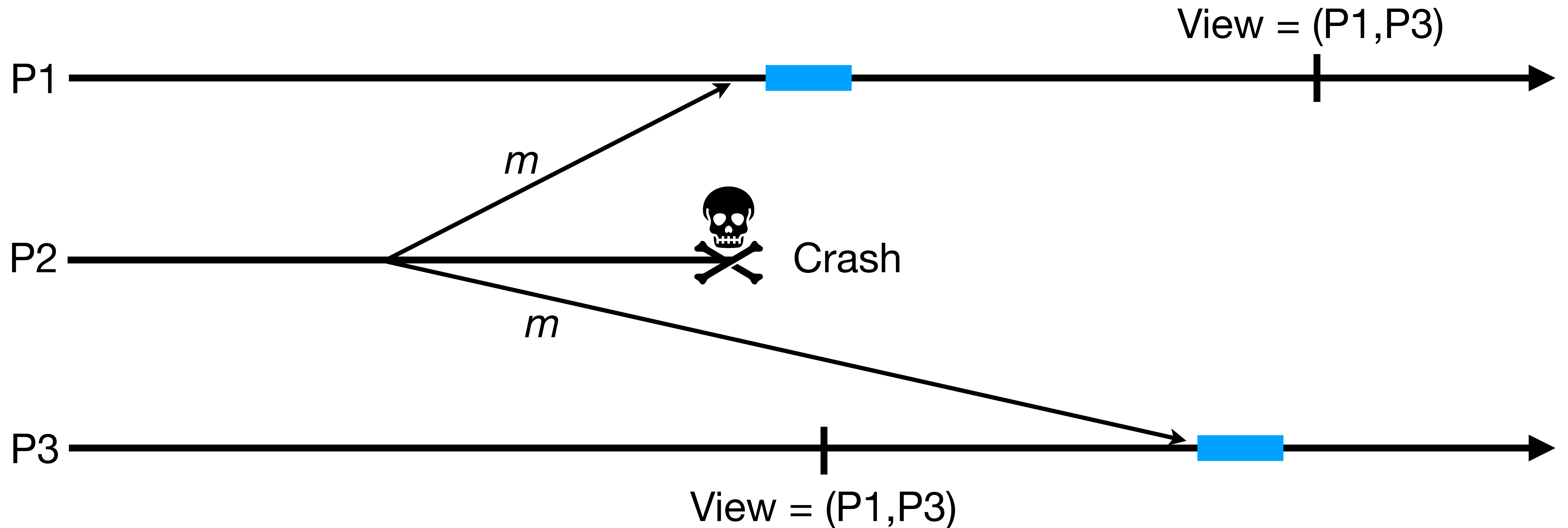


View Synchronous Communication

Motivation

- View Synchronous Communication (VSC) combines:
 - Reliable Broadcast (RB)
 - Group Membership (GM)
- Goal:
 - Ensure **message delivery is ordered with respect to view changes**

Group Membership and Broadcast



The Problem

- P2 broadcasts m
 - P1 delivers m **before** the view change
 - P3 delivers m **after** the view change
 - ⚠ Message delivery *crosses the view boundary inconsistently*
- Combining GM and RB does not prevent this!

Conceptual Tension

- Two conflicting goals:
 - **1** Reliability: We want to deliver m
 - **2** View Consistency: We may want to discard m
- We cannot satisfy both without additional structure

View Synchrony: Core Property

- If a correct process delivers m before installing view V_i ,
 - then every process that installs V_i must deliver m before installing V_i .
- Message delivery is *ordered with respect to view installation*

View Synchronous Broadcast: Intuition

- *Gives the illusion that failures occur atomically with respect to delivered messages.*
- From the application's perspective:
 - Either everyone sees m in the old view
 - Or nobody sees m in the old view
- **Failures appear synchronized.**

View Synchronous Broadcast: A Problem

- What if processes keep broadcasting messages continuously?
- Then:
 - View change *may never* complete
 - *New messages keep appearing in the old view*

View Synchronous Broadcast: Flow Control Mechanism

- Two events:
 -  Block: VSC Module: **Stop broadcasting** messages in this view
 -  BlockOk: Application: **Confirm** it **will not broadcast** more messages in this view
- After BlockOk, the system can **safely move to the next view**

Ch 6.8 View Synchronous Communication

Module 6.9: Interface and properties of view-synchronous communication

Module:

Name: ViewSynchronousCommunication, **instance** vs .

Events:

Request: $\langle vs, Broadcast \mid m \rangle$: Broadcasts a message m to all processes.

Indication: $\langle vs, Deliver \mid p, m \rangle$: Delivers a message m broadcast by process p .

Indication: $\langle vs, View \mid V \rangle$: Installs a new view $V = (id, M)$ with view identifier id and membership M .

Indication: $\langle vs, Block \rangle$: Requests that no new messages are broadcast temporarily until the next view is installed.

Request: $\langle vs, BlockOk \rangle$: Confirms that no new messages will be broadcast until the next view is installed.

Properties:

VS1: View Inclusion: If some process delivers a message m from process p in view V , then m was broadcast by p in view V .

View Synchronous Broadcast: Algorithm Overview

- Each process:
 - Keeps track of all *messages seen in the current view*
 - Detects pending view changes
 - Initiates *flushing* protocol

View Synchronous Broadcast: Algorithm Overview

- When a **new view is pending**:
 - Trigger **Block** to the application
 - Wait for **BlockOk**
 - Start **one TRB instance per process**
 - Each process broadcasts:
 - *The set of messages it delivered in the old view*

View Synchronous Broadcast: Flushing the Old View

- Goal: *Ensure all processes agree on messages from the old view*
- **During the TRB phase:**
 - If some process delivered more messages,
 - Those messages are **delivered to everyone**
 - **Before** installing the new view

Algorithm 6.8: TRB-Based View-Synchronous Communication (part 1, data transmission)

Implements:

ViewSynchronousCommunication, **instance** *vs*.

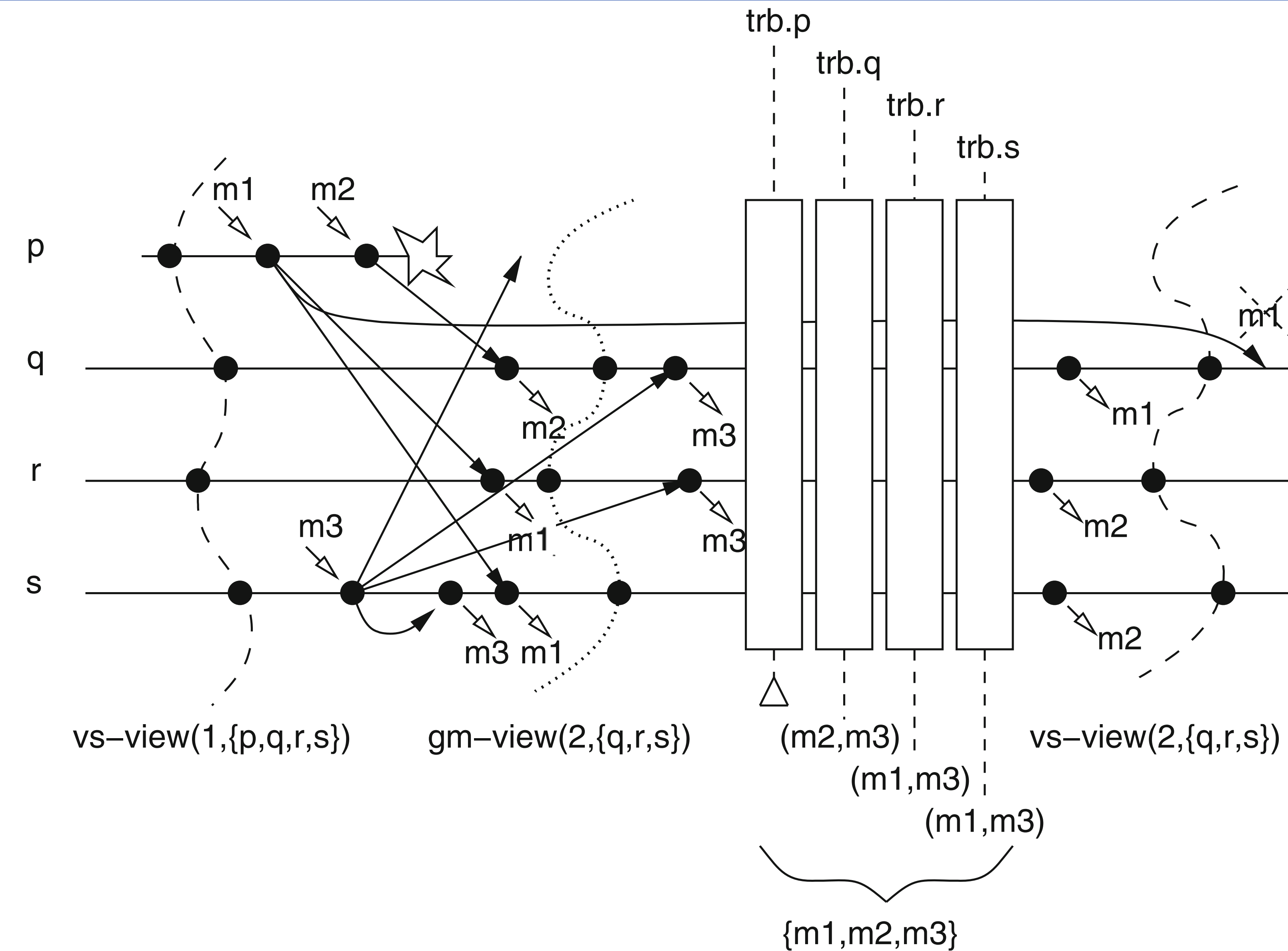
Uses:

UniformTerminatingReliableBroadcast (multiple instances);

GroupMembership, **instance** *gm*;

BestEffortBroadcast, **instance** *beb*.

Ch 6.8 View Synchronous Communication



Algorithm 6.8: TRB-Based View-Synchronous Communication (part 1, data transmission)

Implements:

ViewSynchronousCommunication, **instance** *vs*.

Uses:

UniformTerminatingReliableBroadcast (multiple instances);

GroupMembership, **instance** *gm*;

BestEffortBroadcast, **instance** *beb*.

upon event $\langle vs, Init \rangle$ **do**

$(vid, M) := (0, \emptyset);$ *//* current view $V = (vid, M)$

flushing := FALSE; *blocked* := TRUE;

inview := $\emptyset$;

delivered := $\emptyset$;

pendingviews := [];

trbdone := $\emptyset$;

upon event $\langle vs, Broadcast \mid m \rangle$ **such that** *blocked* = FALSE **do**

inview := *inview* $\cup \{(self, m)\}$;

delivered := *delivered* $\cup \{m\}$;

trigger $\langle vs, Deliver \mid self, m \rangle$;

trigger $\langle beb, Broadcast \mid [DATA, vid, m] \rangle$;

upon event $\langle beb, Deliver \mid p, [DATA, id, m] \rangle$ **do**

if $id = vid \wedge blocked = FALSE \wedge m \notin delivered$ **then**

inview := *inview* $\cup \{(p, m)\}$;

delivered := *delivered* $\cup \{m\}$;

trigger $\langle vs, Deliver \mid p, m \rangle$;

Algorithm 6.9: TRB-Based View-Synchronous Communication (part 2, view change)

```

upon event  $\langle gm, View \mid V' \rangle$  do
    if  $V' = (0, M')$  for some  $M'$  then                                     // start the initial view
         $(vid, M) := (0, M')$ ;
         $blocked := \text{FALSE}$ ;
    else
         $append(pendingviews, V')$ ;

upon  $pendingviews \neq [] \wedge flushing = \text{FALSE}$  do                       // start collective flush procedure
     $flushing := \text{TRUE}$ ;
    trigger  $\langle vs, Block \rangle$ ;

upon event  $\langle vs, BlockOk \rangle$  do
     $blocked := \text{TRUE}$ ;
    forall  $p \in M$  do
        Initialize a new instance  $utrb.vid.p$  of uniform terminating reliable
            broadcast with sender  $p$ ;
        if  $p = self$  then
            trigger  $\langle utrb.vid.p, Broadcast \mid inview \rangle$ ;

    upon event  $\langle utrb.id.p, Deliver \mid p, iv \rangle$  such that  $id = vid$  do
         $trbdone := trbdone \cup \{p\}$ ;
        if  $iv \neq \Delta$  then
            forall  $(s, m) \in iv$  such that  $m \notin delivered$  do
                 $delivered := delivered \cup \{m\}$ ;
                trigger  $\langle vs, Deliver \mid s, m \rangle$ ;

    upon  $trbdone = M \wedge blocked = \text{TRUE}$  do                               // ready to install new view
         $V := head(pendingviews)$ ;  $remove(pendingviews, V)$ ;
         $(vid, M) := V$ ;
         $correct := correct \cap M$ ;
         $flushing := \text{FALSE}$ ;  $blocked := \text{FALSE}$ ;
         $inview := \emptyset$ ;
         $trbdone := \emptyset$ ;
        trigger  $\langle vs, View \mid (vid, M) \rangle$ ;

```

